National University of Computer and Emerging Sciences

Fast School of Computing Fall 2026

CS-1002 – PF (CS Department)

Assignment 3

Section: All sections

Submission Deadline: 11:59 PM. 15th November.

Submission Instructions:

1. Assignments are to be done individually. You must complete this assignment by yourself. You cannot work with anyone else in the class or with someone outside of the class for this assignment. The code you write must be your own, and you must understand each part of it. You are encouraged, however, to get help from the instructional staff through Google Classroom.
2. **Plagiarism:** Plagiarism of any kind (copying from others, copying from the internet or using AI, etc.) is not allowed. If found plagiarised, you will be awarded zero marks in the whole assignment category. Repeating such an act can lead to strict disciplinary actions and failure in the course. Do note that AI and plagiarism will be thoroughly checked across the batch and different LLMs. If any hint of similarity is found, strict action will be taken.
3. A demo for this assignment will be held, where plagiarism and solutions to the assignment's questions will be assessed, and marks for the assignment will be given accordingly.
4. Please start early; otherwise, you will struggle with the assignment.
5. Keep an eye out for the comments on the assignment. Sometimes queries are asked in

the comments, which add constraints to the questions of the assignment.

6. All queries must be asked only in the Google Classroom comment section for the assignment, allowing others with similar questions to be able to have the answers to their questions.

7. If you have a query, kindly check if anyone has already asked a similar query beforehand.

8. A submission portal will be uploaded for you to submit the assignment.

9. Submit the .cpp files on the portal only within the deadline. Submissions on emails will not be accepted.

10. Try to submit at-least 1 hour before the deadline to tackle potential internet issues regarding the submission.

11. Submissions other than the portal(e.g. email, GCR, etc.) will not be accepted.

12. You are supposed to write a function for each question of the assignment. In main() you are supposed to make a switch statement to run each question and you have to call each function in their respective case.

13. Remember only main() is a menu-driven switch with one case per question. Inside each case, follow that question's own restrictions.

14. The student is solely responsible for checking the final submission for issues like corrupt files sent. If we cannot download the file for any reason, it will lead to zero marks in the assignment

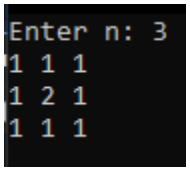
15. This assignment tests Loops, Functions, and Arrays. You are not allowed to use recursion (directly or indirectly), STL containers like std::vector, std::array, or std::string etc.

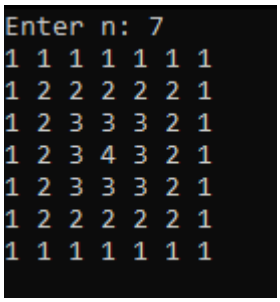
Note: Follow the given instructions; failing to do so may result in a zero in this assignment.

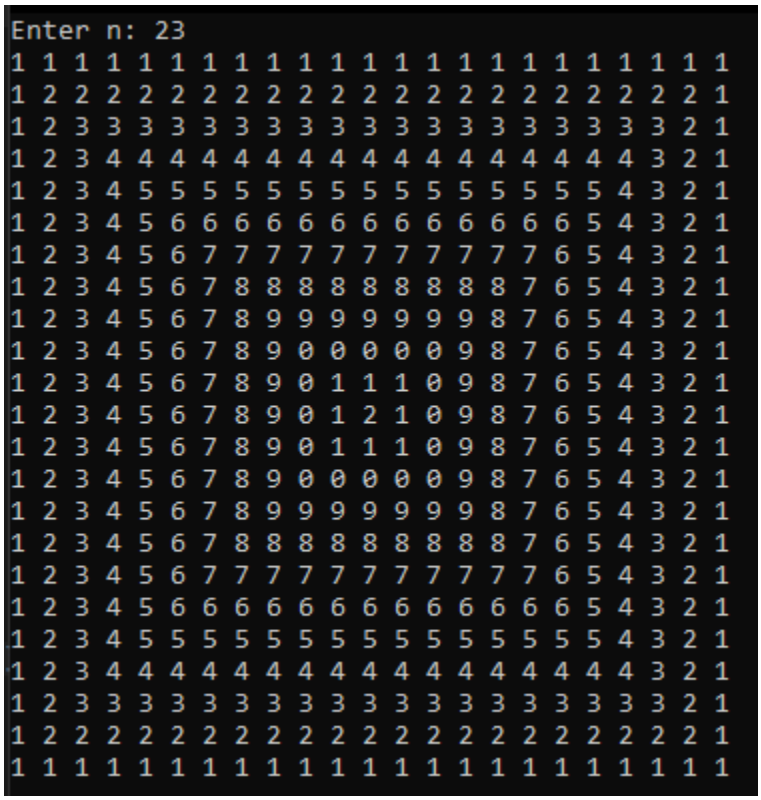
Question 1 (15 Marks)

Analyze the sample outputs given below and write a C++ program that generates the same pattern with the following constraint enforced:

- The input n must be an odd number greater than 2.





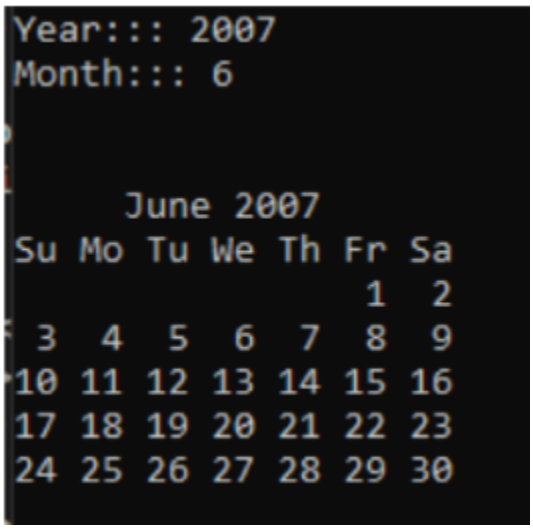


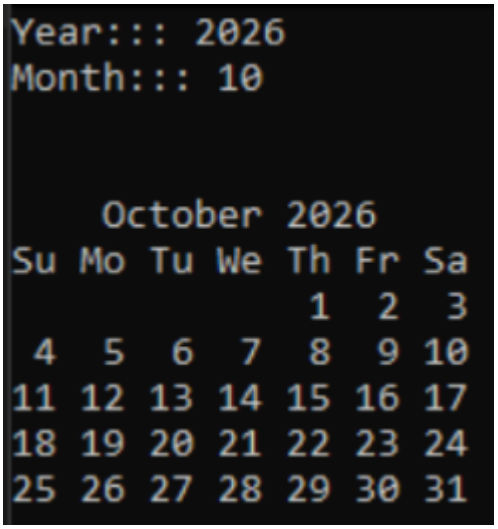
Question 2 (25 Marks)

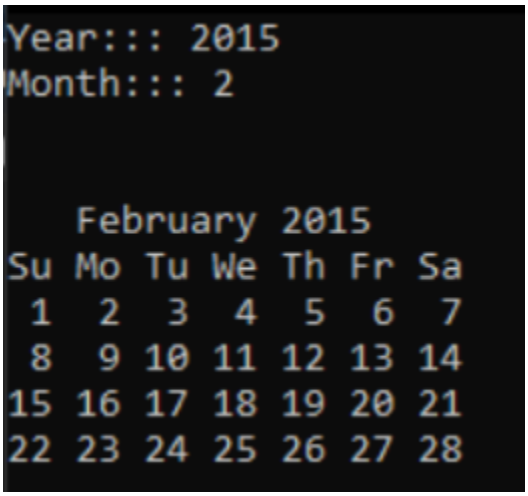
Write a C++ program that reads a **year** and a **month** and prints the calendar of that month.

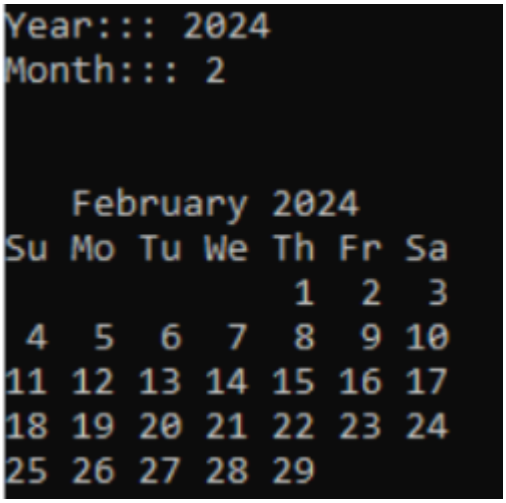
Rules:

- The year must be between 1 and 9999.
- Use the Gregorian calendar for all years.
- Do not use any date or time library, and do not use arrays. Use loops only.







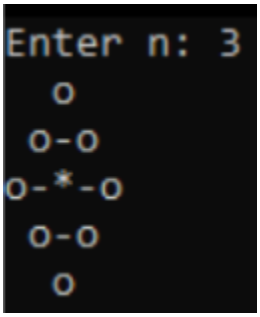


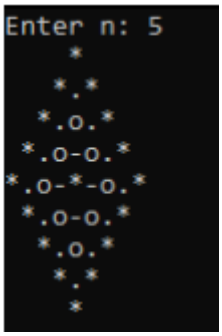
Question 3 (30 Marks)

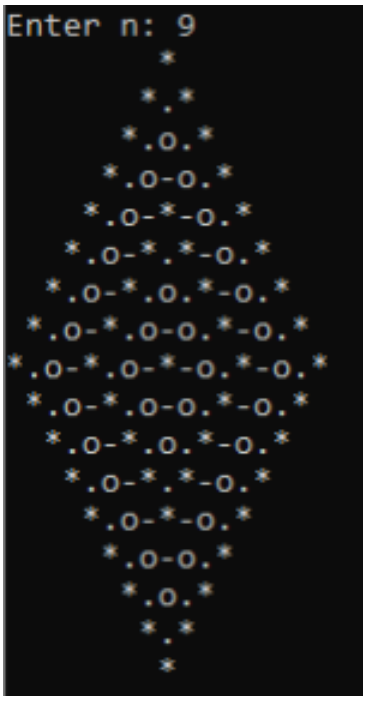
Write a c++ program that prints the following pattern based on the value of the input.

Following constraints need to be enforced:

- Input needs to be greater than 2.
- Input can never be even.
- Use **FOR** loops only.







Question 4: Ripple Pond (35 Marks)

Analyze the sample outputs given below and identify the formulas and relations, then write a C++ program that generates the same patterns for any N, Base and Seed input. You are **not** allowed to use arrays. The program must display the pattern numbers in the user-specified number system (from binary till hexadecimal).

Example Outputs:

```
N: 4
Base: 10
Seed: 5

9 9 9 9 9 9 9
5 7 7 7 7 7 9
3 5 6 6 6 7 9
1 3 4 5 6 7 9
1 3 4 4 4 5 7
1 3 3 3 3 3 5
1 1 1 1 1 1 1
```

```
N: 5
Base: 16
Seed: 7

E E E E E E E E E
8 B B B B B B B E
4 7 9 9 9 9 9 B E
2 5 7 8 8 8 9 B E
0 3 5 6 7 8 9 B E
0 3 5 6 6 6 7 9 C
0 3 5 5 5 5 5 7 A
0 3 3 3 3 3 3 3 6
0 0 0 0 0 0 0 0 0
```

```
N: 8
Base: 16
Seed: 3

4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
A 7 7 7 7 7 7 7 7 7 7 7 7 7 7 4
A 7 F F F F F F F F F F F F 7 4
0 D 5 A A A A A A A A A A F 7 4
A 7 F 4 7 7 7 7 7 7 7 A F 7 4
6 3 B 0 3 5 5 5 5 5 7 A F 7 4
4 1 9 E 1 3 4 4 4 5 7 A F 7 4
2 F 7 C F 1 2 3 4 5 7 A F 7 4
2 F 7 C F 1 2 2 2 3 5 8 D 5 2
2 F 7 C F 1 1 1 1 1 3 6 B 3 0
2 F 7 C F F F F F F F 2 7 F C
2 F 7 C C C C C C C C C 1 9 6
2 F 7 7 7 7 7 7 7 7 7 7 7 F C
2 F F F F F F F F F F F F F C
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Question 5: Lost In The Mine (40 Marks)

Problem

A miner is trapped in a square mine of size n × n. Write a program that reads the mine from the terminal and reports:

1. whether the miner can reach the exit, and
2. how many gold cells the miner can reach from the start.

You may NOT use arrays. This includes strings, vectors, or any other container.

Input

- First, an integer n ($2 \leq n \leq 10$).
- Then n × n characters, row by row, left to right, separated by spaces or newlines.

Character	Meaning
`S`	Start (exactly one)
`E`	Exit (exactly one)
`.`	Empty
`*`	Wall
`G`	Gold

Movement

- The miner moves up, down, left, or right.
- The miner cannot enter a wall or leave the grid.
- `S`, `E`, `G` and `.` can all be walked on.

Output

Print exactly two lines:

```
Solvable: YES   (or NO)
Gold reachable: k
```

Example 1

Input:

```
5
S . * . .
* . * G .
* . . * .
G * . . .
. . * * E
```

Output:

```
Solvable: YES
Gold reachable: 1
```

Example 2

Input:

```
4
S . * .
G * . .
. . * *
* . * E
```

Output:

```
Solvable: NO
Gold reachable: 1
```

Question 6: CRUEL ANGEL'S THESIS - NERV Systems Recovery Division (45 Marks)

Background Story

The latest Angel attack has left Tokyo-3 in ruins, and NERV has suffered catastrophic damage to its computer infrastructure. The central operating system responsible for managing NERV's computational resources has been corrupted beyond recovery.
The hardware is still functional. The memory modules are intact, the CPU is operational, and the command center is barely holding together. However, without an operating system, NERV can no longer manage its processes, schedule computational tasks, or allocate memory efficiently.

The engineers have determined that rebuilding the original operating system is impossible within the available time. As the newest programmer assigned to the Systems Recovery Division, you have been ordered to develop a replacement system from scratch.

Your system will manage NERV's computational processes, schedule their execution, and allocate the limited memory available to the command center. A failure in resource management could delay critical operations during the next Angel attack.

Your mission is to rebuild the core functionality of NERV's operating system before the next Angel arrives.

Project Requirements

1. Process Management System

NERV requires a system capable of managing up to 100 computational processes.

Each process must maintain the following information:

- Process ID (PID), assigned uniquely by the system
- Arrival time
- Burst time
- Remaining burst time

- Priority
- Completion time
- Turnaround time
- Waiting time
- Memory requirement
- Current process status

Your system must provide the following functionalities:

1. Register a new process.
2. Display all registered processes and their information.
3. Search for a process using its PID.
4. Display processes based on their current status.
5. Terminate a process and release its allocated resources.
6. Display the execution and performance statistics of completed processes.

Processes must be tracked through appropriate states, including New, Ready, Running, and Terminated.

2. CPU Scheduling

The recovered system must support three CPU scheduling algorithms.

2.1 First Come, First Served (FCFS)

Processes must be scheduled in ascending order of arrival time. A process, once selected, executes until its burst time is exhausted.

2.2 Shortest Job First (SJF)

Among the processes eligible for execution, the process with the shortest burst time must be selected. This algorithm must be non-preemptive.

2.3 Round Robin (RR)

Each eligible process receives a fixed time quantum. If a process does not finish during its allocated quantum, its remaining burst time must be updated, and it must return to the end of the ready queue.

For each algorithm, your system must calculate and display:

- Completion time (CT)
- Turnaround time (TAT)
- Waiting time (WT)
- Average turnaround time
- Average waiting time
- CPU utilization

The system must also display the order in which processes execute and their final scheduling results.

Important: Scheduling algorithms are to operate on the registered set of processes and calculate their schedules. You are not required to build a real-time, continuously advancing operating system simulation.

3. Memory Management

NERV's central computer has a fixed memory capacity of 100 memory blocks, with each block representing one unit of memory.

You must implement memory management using arrays.

3.1 First Fit Allocation

Implement the First Fit memory allocation algorithm.

When a process requests memory, the system must scan memory from the beginning and identify the first contiguous sequence of free blocks large enough to satisfy the request.

If sufficient contiguous space is available, allocate the required blocks to the requesting process. Otherwise, the request must fail.

3.2 Memory Deallocation

When a process terminates, all memory blocks allocated to it must be released.

The system must maintain an accurate record of memory ownership and display the current memory layout on request.

3.3 Memory Fragmentation

The system must account for external fragmentation.

A process's memory request may fail even when the total amount of free memory is sufficient, because the free memory is divided into smaller, non-contiguous regions.

The system must be able to report:

- Total free memory
- Total allocated memory
- Size of the largest contiguous free region
- Whether a requested allocation can currently be satisfied

3.4 Memory Compaction

Repeated allocation and deallocation can fragment NERV's memory, potentially preventing new processes from being loaded.

Implement a memory compaction operation that reorganizes memory to combine scattered free blocks into a single contiguous free region.

Your implementation must:

1. Move allocated memory regions toward the beginning of memory.
2. Preserve the relative order of allocated processes.
3. Preserve the memory size allocated to each process.
4. Update the memory layout after compaction.
5. Ensure that no allocation is lost, duplicated, or assigned to the wrong process.
6. Make the resulting free space available for subsequent allocation requests.

The system must display the memory layout after compaction and report the resulting contiguous free space.

4. Integrated System Operations

The process management, CPU scheduling, and memory management components must function as parts of one system.

The following operations must be supported:

- Register a process with its execution and memory requirements.
- Request memory for a process.
- Schedule processes using any supported scheduling algorithm.
- Update remaining burst times during scheduling.
- Mark processes as completed when their execution finishes.
- Release memory when a process terminates.
- Compact memory when fragmentation prevents a request from succeeding.
- Display the current state of processes and memory.

A process whose memory allocation fails must not be treated as successfully loaded. The system must maintain consistency between process records and memory ownership.

5. NERV Diagnostics and Statistics

The command center requires a diagnostic report containing:

- Total registered processes
- Number of completed processes
- Number of ready processes
- Number of running processes
- Total memory capacity
- Total allocated and free memory
- Largest contiguous free memory region
- Successful memory allocation requests
- Failed memory allocation requests
- Scheduling statistics for completed processes

All statistics must be calculated from the system's actual records.

6. Command Interface

The system must provide a menu-driven interface that allows operators to perform the required tasks without restarting the program.

The interface must include options for:

- Process registration and searching
- Process and status display
- CPU scheduling algorithm selection
- Memory allocation and deallocation
- Memory layout display
- Memory compaction
- System statistics
- Program termination

Use separate functions for logically distinct operations. The system should be modular, and repeated logic should not be unnecessarily duplicated.

Implementation Constraints

The replacement operating system must be developed under the following restrictions:

- Use C++.
- Only #include <iostream> is permitted.
- Use arrays, loops, conditional statements, and functions.
- Do not use additional libraries or built-in container utilities.
- Do not use dynamic memory allocation.
- Do not use global variables.
- Implement the required scheduling and memory management algorithms yourself.

The submitted program must integrate all the required components into a single working system.

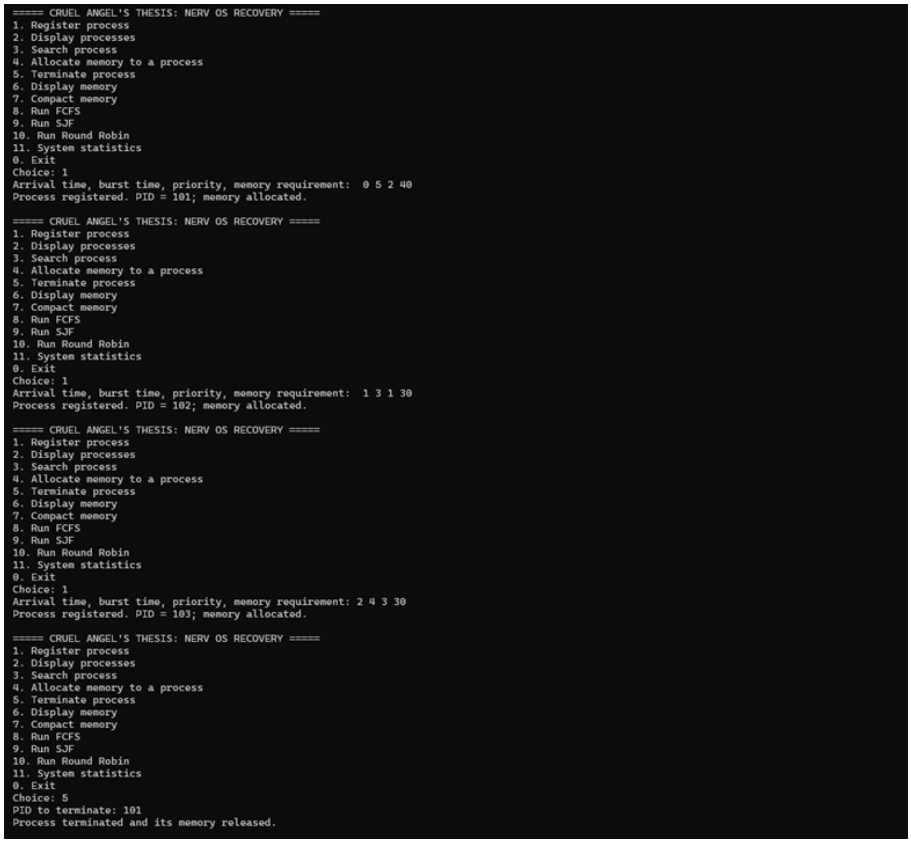
Final Directive

NERV's engineers have restored the hardware, but the next Angel will not wait for the software to be ready. Your task is to build a system that works correctly under normal operating conditions and maintains consistent records as processes are registered, scheduled, completed, and removed.

A system that merely displays the correct output for one demonstration is insufficient. The implementation must correctly manage process information, CPU scheduling results, and memory ownership throughout its operation.

The survival of Tokyo-3 depends on your ability to restore the system. Good luck.

SAMPLE OUTPUT



```
===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 5
PID to terminate: 103
Process terminated and its memory released.

===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 11
Registered processes: 3
Completed: 2
Ready: 1
Running: 0
Waiting for memory: 0
Memory capacity: 100
Allocated memory: 30
Free memory: 70
Largest free region: 40
Successful allocations: 3
Failed allocations: 0

===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 1
Arrival time, burst time, priority, memory requirement: 3 6 2 50
Process registered as NoMemory. Allocation failed; try compaction or retry allocation later. PID = 104
```

```
===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 4
PID to allocate memory for: 104
Allocation failed. Free memory = 70; largest region = 40

===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 7
Memory compaction completed. Relative order of allocated blocks was preserved.

Memory layout (each symbol is one block):
[102][102][102][102][102][102][102][102][102][102]
[102][102][102][102][102][102][102][102][102][102]
[102][102][102][102][102][102][102][102][102][102]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
[ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ][ . ]
Free blocks: 70
Largest contiguous free region: 70

===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 4
PID to allocate memory for: 104
Memory allocation successful. Process is Ready.

===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 8
```

```
Execution order: 102(3) 104(6)

PID     CT      TAT     WT
102     4       3       0
104     10      7       1
Completed in this schedule: 2
Average waiting time: 0.5
Average turnaround time: 5
CPU utilization: 90%
This is a scheduling analysis; process states and memory allocations are unchanged.

===== CRUEL ANGEL'S THESIS: NERV OS RECOVERY =====
1. Register process
2. Display processes
3. Search process
4. Allocate memory to a process
5. Terminate process
6. Display memory
7. Compact memory
8. Run FCFS
9. Run SJF
10. Run Round Robin
11. System statistics
0. Exit
Choice: 0
NERV OS recovery simulator shutting down.
```

Question 7: CONNECT 4 (45 Marks)

You are building the brain of a Connect 4 opponent. A human plays against your program: after each human move, your program looks a few moves ahead, assumes the human will always reply in the way that is worst for it, and plays the column that works out best under that assumption. How to play connect 4

Game Rules

- The board has 6 rows and 7 columns, numbered 0 to 6. A piece falls to the lowest empty cell of its column.
- The human (X) moves first. The computer (O) moves second.
- Four in a row (horizontal, vertical or diagonal) wins. A full board is a draw.

How the Computer Thinks

Count each piece placed during the look-ahead as one *ply*. The computer's own move is ply 1, the human's reply is ply 2, and the computer's follow-up is ply 3. The search stops after **ply 3**.

1. The computer tries each column it is allowed to play, then each reply the human could make, then each follow-up of its own.
2. At the human's turn, assume the human picks whatever is worst for the computer. At the computer's turn, it picks whatever is best for itself. The blocked-column rule applies at every ply of the look-ahead.
3. While exploring, keep two numbers for the position you are currently looking at:
    - **L**: the best score the computer is already sure to get from the options explored so far.
    - **U**: the lowest score the human is already sure to force from the options explored so far.

    At the very start, L = -1000000 and U = +1000000. Whenever you try a column and look deeper, the deeper position starts with the same L and U as the position you came from. Once you have the score for a column you tried:

    - If it was the **computer's** turn, L becomes the larger of L and the best score found so far at this position.
    - If it was the **human's** turn, U becomes the smaller of U and the best score found so far at this position.

As soon as L >= U, stop trying the remaining columns at this position.

4. **Order of columns:** always try columns in the order 3, 2, 4, 1, 5, 0, 6.
5. **Ties:** the computer replaces its best column only when a new score is strictly greater, so the first best column found is kept.
6. **A win or draw at ply 1 or 2 ends that branch.** The search does not go deeper from there, and the node is scored immediately (+100002 at ply 1, -100001 at ply 2, 0 for a full board).

Scoring (from the computer's point of view)

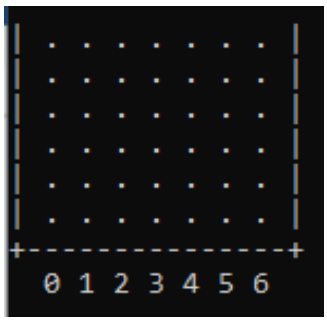
- Computer wins at ply p: +(100000 + 3 - p)
- Human wins at ply p: -(100000 + 3 - p)
- Board full: 0
- If ply 3 is reached with no win or draw, look at every group of 4 consecutive cells (horizontal, vertical and both diagonals):

Group contents	Score
3 computer + 1 empty	+5
2 computer + 2 empty	+2
3 human + 1 empty	-4
any other mix	0

Then add +3 for every computer piece in column 3.

show the board after every move, reject invalid, full or blocked columns and ask again, and announce a win or a draw.

Empty board:



After player has inserted in C0:



Question 8: PTERO, the Cave Flyer (65 Marks)

Overview

Build **PTERO** as shown in the gameplay video. Only the cave on screen exists: anything that scrolls off is forgotten, so flying back shows new cave.

Allowed: variables, `if`/`else`, `switch`, loops, functions, pass by reference, 1D and 2D arrays, `const` variables inside functions, `<iostream>` and `terminal.h`.

Not allowed: global variables or constants of any kind; structs, classes, pointers, `new`/`delete`, `std::string`, `std::vector`; any other header; `rand()`, `srand()`, `time()` or any maths library function.

You write `ptero.cpp`. `terminal.h` is given; do not edit it. The game is marked on Ubuntu.

Screen and controls

- 116 × 33 characters: a 114 × 30 playfield inside a border, with the HUD on the last line.
- Rows 0 and 29 of the playfield are always rock.
- W/Up and S/Down move one row. A/Left and D/Right face that way. R/Space plays again (game over only). Q quits at any time.
- During the crash and on the game over screen, every key except R, Space and Q is ignored.

The bird (Pterodactyl)

Use these exact characters:

```
facing right              facing left
wings up    wings down    wings up    wings down

   |\                         /|
~==|_\o=>   ~==|-/o=>     <=o/_|==~   <=o\-|==~
              |/                          \|
```

- The sprite's left edge is always at screen column 53.
- Spaces in the sprite are see-through: they are not drawn and cannot crash.
- Wing state = `(frame / 3) % 2`.
- The bird moves one column every frame in the direction it faces, and cannot stop.

The cave

The cave is random, but it is made by exact rules. The walls are built from straight **stretches**, and every new column is made from the column next to it.

What is stored

The cave is two arrays with one entry per screen column, 114 each:

- `caveTop[c]`: the first open row of column c. Every row above it is rock.
- `caveBottom[c]`: the first rock row of the floor. It and every row below it are rock.
- The **gap** of a column is `caveBottom[c] - caveTop[c]`, the number of open rows.

Rows 0 and 29 are always rock, so `caveTop` is at least 1 and `caveBottom` at most 29.

Only these 114 columns exist. Every frame the cave scrolls one column: the column that leaves the screen is forgotten, and one new column is made at the **leading edge**, the side the bird flies towards. It is made from its **neighbour**, the column next to it on the screen:

Flying	New column	Made from
Right	113	112
Left	0	1

Stretches

A stretch has a **direction** and a **length**:

- direction −1: the cave moves up one row per column;
- direction 0: it stays level;
- direction +1: it moves down one row per column;
- length: how many columns of the stretch are still to be made.

When a new column is needed and the current stretch has no columns left, a new stretch starts:

1. If the bird has flown at least 150 columns, it may be a rock stretch (see Rocks).
2. Otherwise it is an ordinary stretch: `direction = randomInt(-1, 1)` and `length = randomInt(6, 25)`.

Making one column

In an ordinary stretch, the new column is made from its neighbour in eight steps. All divisions are integer divisions.

1. `gap = caveBottom[old] - caveTop[old]` and `centre = caveTop[old] + gap / 2`.
2. Move the middle: `centre = centre + direction`.
3. Let the gap breathe. Take `roll = randomInt(0, 99)`:
    - 0 to 7: `gap` goes up by 1;
    - 8 to 15: `gap` goes down by 1;
    - 16 to 99: no change.

4. Keep `gap` between `minGap` and `minGap + 4`.
5. `top = centre - gap / 2`.
6. Keep the opening on screen:
    - if `top < 1`, set `top = 1`;
    - if `top + gap > 29`, set `top = 29 - gap`;
    - if either one happened, set `direction = 0` for the rest of the stretch.
7. Store `caveTop[new] = top` and `caveBottom[new] = top + gap`.
8. Take 1 off the stretch's length.

A column in a rock stretch skips steps 1 to 6: it copies its neighbour's `caveTop` and `caveBottom` exactly.

Example: the neighbour has `caveTop` 5 and `caveBottom` 25, so `gap` is 20 and `centre` is 15. With direction +1, `centre` becomes 16. The roll is 42, so the gap stays 20, which is inside 18 to 22. Then `top = 16 - 10 = 6`, and the new column has `caveTop` 6 and `caveBottom` 26.

Getting narrower

`minGap = 18 - distance / 250`, but never less than 9. `distance` is the value shown on the HUD. The cave starts with gaps of 18 to 22 rows, loses one row every 250 columns, and is 9 to 13 rows wide from 2,250 columns on.

The first frame

1. Column 57 gets `caveTop` 5 and `caveBottom` 25: centre row 15, gap 20.
2. Start a new stretch, then make columns 58, 59, … 113, each from the column on its left.
3. Start a new stretch, then make columns 56, 55, … 0, each from the column on its right.
4. Start a new stretch when the bird starts flying.

The distance is 0 here, so the first frame has no rocks. **R** builds a fresh cave the same way.

Turning round

When the bird turns, the stretch in progress is dropped. The first stretch at the new leading edge is ordinary, flat (direction 0) and exactly 10 columns long; its gap still breathes. After it, stretches start as usual.

Rocks

```
small, 3 x 6     large, 4 x 8
 .--.                __
(####)             _/##\_
 `--'             (######)
                   `----'
```

- After 150 columns, a new stretch is a **rock stretch** if `randomInt(0, 99) < 35 + 5 × level`, capped at 70.
- A rock stretch is flat, exactly 28 columns long, and keeps its gap unchanged.
- Rock type = `randomInt(0, 1)` (0 small, 1 large). Its top row = `randomInt(top + 1, bottom - 1 - height)`.
- If that leaves fewer than 4 free rows both above and below, the rock moves up to `top + 1`.
- The rock sits in the middle of the stretch, from column `(28 - width) / 2`. It must read the right way round in both directions.
- Rocks scroll with their columns.

Crashes and scoring

- After moving, check the sprite as it will be drawn. Any non-space character on cave rock or a floating rock is a crash.
- **Distance:** +1 for every frame flown in this life, in either direction.
- **Best:** the largest distance since the program started.
- **Frame wait:** `70 - distance / 40` ms, never below 30.
- **HUD:** `DIST` and `BEST` as 5 digits with leading zeros. `SPEED` is 10 characters: one `>` per 4 ms the wait has dropped below 70, then `-`. `FLYING` shows `>>` or `<<`.

terminal.h

`terminal.h` gives you everything that would otherwise need another header: reading a key without stopping the program, redrawing the screen in place, timing and random numbers. Put it next to `ptero.cpp`, write `#include "terminal.h"` at the top, and do not change it. Your own code must not include any header except `<iostream>`.

Function	What it does
`termInit()`	Call once, just before the game loop. Clears the screen and hides the cursor. From now on keys arrive without Enter and are not shown on screen.
`termRestore()`	Call once, before the program ends. Puts the terminal back to normal and shows the cursor. Ctrl+C also restores it.
`keyPressed()`	`true` if a key is waiting. Returns at once and never waits, so the game keeps running.
`readKey()`	Returns the waiting key: a lowercase letter (`'w'` even if Shift or Caps Lock is on), or one of `KEY_ARROW_UP`, `KEY_ARROW_DOWN`, `KEY_ARROW_LEFT`,

Function	What it does
	`KEY_ARROW_RIGHT`. Only call it after `keyPressed()` returns `true`.
`flushKeys()`	Throws away any keys still waiting. Call it when the game over box appears, so a key held down during the crash does not skip the box.
`cursorHome()`	Moves the cursor to the top-left corner, so the next frame prints over the last one instead of below it.
`sleepMs(ms)`	Waits `ms` milliseconds.
`randomInt(low, high)`	A random whole number from `low` to `high`, both included. Use it instead of `rand()`. It starts from the clock, so every run makes a different cave; you never seed anything yourself.

A minimal program using it:

```
#include <iostream>
#include "terminal.h"
using namespace std;

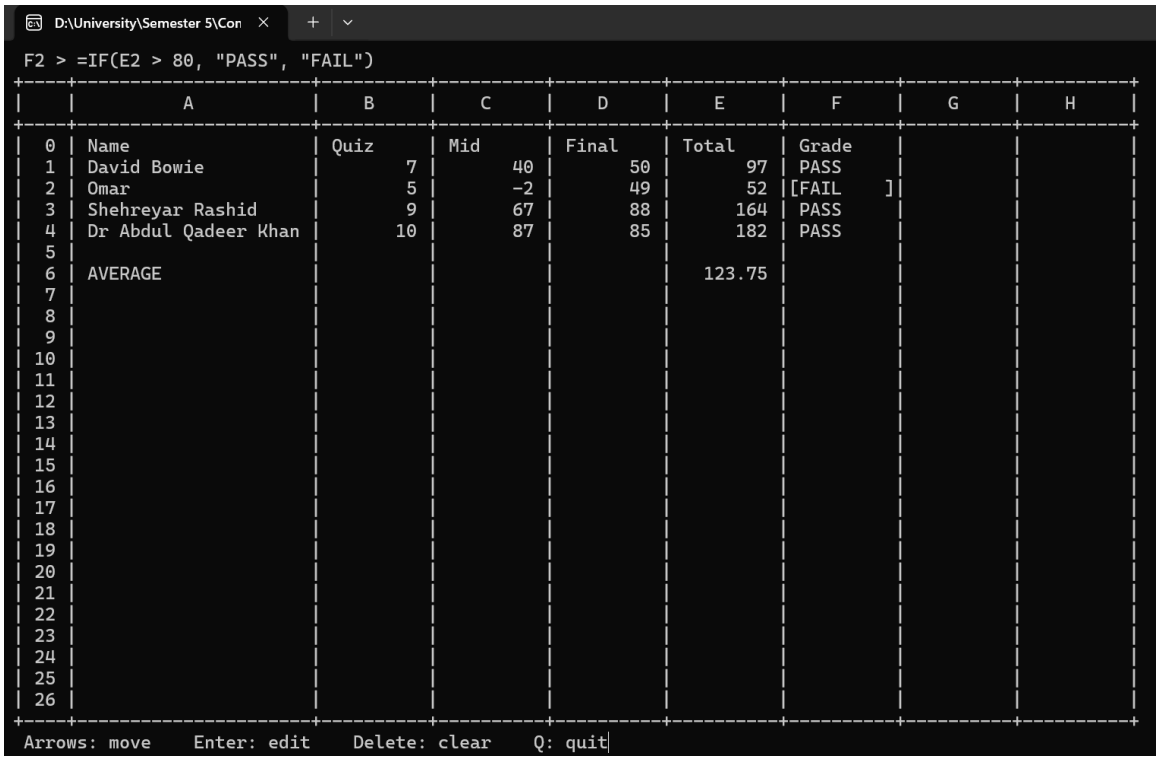
int main()
{
    termInit();
    while (true)
    {
        while (keyPressed())
        {
            int key = readKey();
            if (key == 'q')
            {
                termRestore();
                return 0;
            }
            if (key == 'w' || key == KEY_ARROW_UP)
            {
                // move up
            }
        }
        int step = randomInt(-1, 1);     // -1, 0 or 1
        // update, then draw: cursorHome() and one cout
        sleepMs(70);
    }
}
```

BONUS QUESTION: CELLS, a Terminal Spreadsheet (100 Marks)

Overview

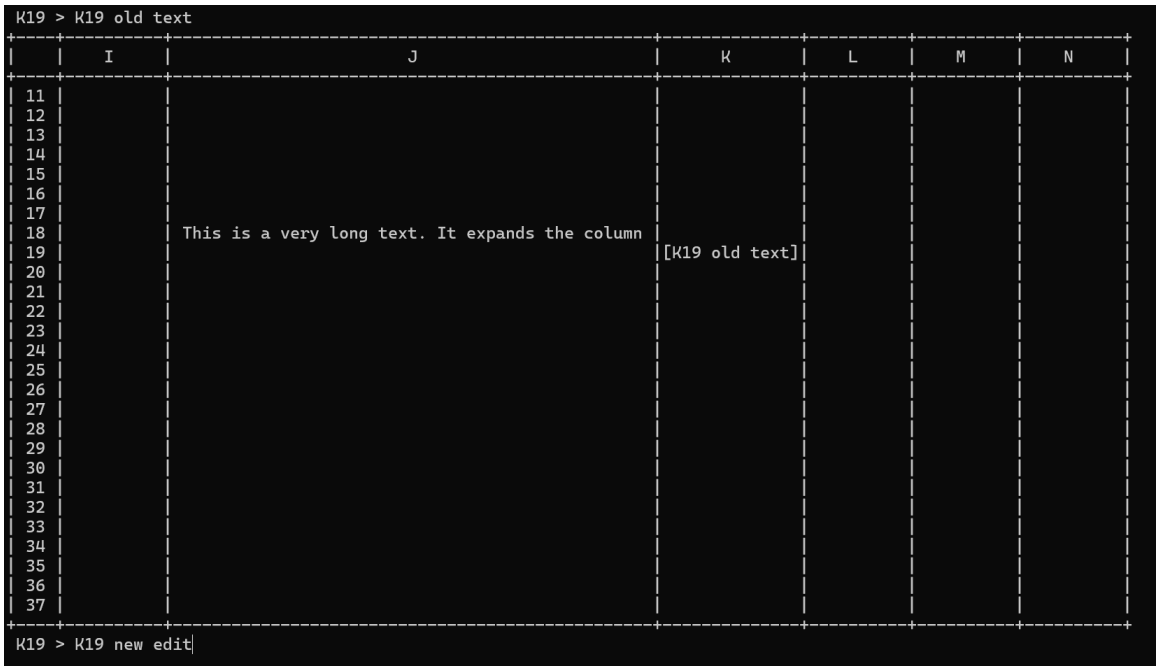
You will build **CELLS**, a spreadsheet that runs in the terminal. The sheet has 26 columns, A to Z, and 51 rows, 0 to 50. You move with the arrow keys and type text, numbers or formulas into cells. Every formula updates as soon as anything it uses changes.

The screen



	A	B	C	D	E	F	G	H
0	Name	Quiz	Mid	Final	Total	Grade		
1	David Bowie	7	40	50	97	PASS		
2	Omar	5	-2	49	52	[FAIL]		
3	Shehreyar Rashid	9	67	88	164	PASS		
4	Dr Abdul Qadeer Khan	10	87	85	182	PASS		
5								
6	AVERAGE				123.75			
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								

- CELLS always draws inside a fixed area **116 characters wide and 33 lines tall**. When the cursor leaves it, the view scrolls by the fewest rows or columns that bring the cursor's cell back. Only whole columns are shown.
- Each column is as wide as the longest shown text in it, but never narrower than 8.



Editing a cell: the formula bar still shows the cell K19's old text, and the new text is typed after the cell's name on the last line.

Input and helperCells.h

Put `helperCells.h` next to `cells.cpp`, write `#include "helperCells.h"` at the top, and do not change it. It gives you two functions:

Function	What it does
`readKey()`	Stops the program until a key is pressed, then returns at once. There is no need to press Enter, and the key is not shown on the screen. It returns `KEY_UP`, `KEY_DOWN`, `KEY_LEFT`, `KEY_RIGHT`, `KEY_ENTER`, `KEY_DELETE` (Delete or Backspace), or for any other printable key the character itself, such as `'q'`. Keys like Esc, Tab and F1 are skipped, and it keeps waiting.
`clearScreen()`	Clears the screen and moves the cursor to the top-left corner. Call it before printing each new screen.

Outside `readKey()`, the terminal behaves normally: what is typed is shown, and `cin` reads it once Enter is pressed. CELLS uses both.

Moving. Call `readKey()` and act on the key:

Key	Action
Arrow keys	Move the cursor one cell. It stops at the edges of the sheet.
Enter	Edit the cursor's cell.
Delete or Backspace	Empty the cursor's cell.
Q or q	Quit.

Editing. Print the screen with its last line replaced by the cell's name and `>`, such as `F2 >`, then read the new text with `cin`. The person types it after the `>` and presses Enter. The line replaces the cell's text and the cursor stays on that cell. Only the first 60 characters are kept, and only printable ones (codes 32 to 126). An empty line leaves the cell as it was.

```
#include <iostream>
#include "helperCells.h"
using namespace std;

int main()
{
    while (true)
    {
        clearScreen();
        // print the whole screen

        int key = readKey();
        if (key == KEY_UP)
        {
            // move the cursor up
        }
        else if (key == KEY_ENTER)
        {
            // print the screen with the edit line last, then read the
new text with cin
        }
        else if (key == 'q' || key == 'Q')
            return 0;
    }
}
```

`helperCells.h` is made for Ubuntu and also works in a Windows console. On Windows, maximise the window first: it opens shorter than 33 lines.

Cells

Typed	The cell is	It shows
Nothing	Empty	Nothing
Starts with `=`	A formula	Its result: a number, a text or an error
Digits, with an optional `-` in front and an optional `.` followed by more digits	A number	The number
Anything else	A text	The text as typed

Numbers are shown rounded to 4 decimal places, halves away from zero, with no trailing zeros: 7, 32.75, 0.3333. A number of 10^15 or more in size shows `#NUM!`.

Formulas

A formula is `=` followed by an expression. Spaces outside quotes are ignored, and cell and function names may be either case. After every edit, every formula shows its up-to-date result.

Part	Examples	Meaning
Number	`12`, `3.75`	That number
Text	`"PASS"`	The characters between the quotes, which cannot include `"`
Cell	`B3`, `z50`	That cell's value
Range	`A1:C3`, `C3:A1`	Every cell in the rectangle, row by row. Only as an argument of a list function.
Function	`ROUND(SUM(A1:A4) / 3, 2)`	Any argument can be any expression.
Brackets	`(1 + 2) * 3`	Worked out first

Order	Operators	Notes
1	`^`	Right to left: `2^3^2` is 512. The power must be a whole number.
2	`-` in front of a value	`-2^2` is −4, `2^-1` is 0.5
3	`*` `/`	Left to right

Order	Operators	Notes
4	`+` `-`	Left to right
5	`=` `<>` `<` `>` `<=` `>=`	Left to right. True gives 1, false gives 0.

- An empty cell gives 0.
- Arithmetic needs numbers.
- Comparisons take two numbers or two texts. Texts compare character by character, by character code; a text that is the start of a longer one is the smaller.
- A formula that uses an error gives that error. If it meets more than one, any of them is accepted.

Errors

A formula that cannot be read shows `#SYNTAX!`, whatever else is wrong with it.

Error	When
`#SYNTAX!`	An unknown character, a missing value or bracket, an unclosed quote, two values with nothing between them, or nothing after `=`
`#NAME?`	A name that is not a function
`#REF!`	A cell outside A0 to Z50
`#VALUE!`	Text in arithmetic, a number compared with a text, a range outside a list function, or the wrong number of arguments
`#DIV/0!`	Dividing by zero, `MOD` by zero, or `AVG` of no numbers
`#NUM!`	A power that is not whole, zero to a negative power, `SQRT` of a negative number, `MEDIAN` of no numbers, or a number of 10^15 or more in size
`#CYCLE!`	A formula that uses its own cell, directly or through other formulas

Functions

A **list** is one or more values or ranges, separated by commas. In `SUM`, `AVG`, `MIN`, `MAX`, `COUNT` and `MEDIAN`, only numbers count: text and empty cells are skipped. The examples use the sheet in the screenshot.

Function	Result	Example
`SUM(list)`	The total. 0 if there are no numbers.	`=SUM(B1:B4, 10)` gives 41
`AVG(list)`	The average. `#DIV/0!` if there are no numbers.	`=AVG(B1:B4)` gives 7.75
`MIN(list)`	The smallest. 0 if there are no numbers.	`=MIN(E1:E4)` gives 52
`MAX(list)`	The largest. 0 if there are no numbers.	`=MAX(E1:E4)` gives 182
`COUNT(list)`	How many numbers there are.	`=COUNT(A0:F4)` gives 16
`MEDIAN(list)`	The middle number once sorted; with an even count, the average of the two middle ones. `#NUM!` if there are none.	`=MEDIAN(B1:B4)` gives 8
`CONCAT(list)`	The shown text of every value, joined in order.	`=CONCAT(A2, " got ", E2)` gives Omar got 52
`ABS(x)`	x without its sign.	`=ABS(C2)` gives 2
`ROUND(x, d)`	x rounded to d decimal places, halves away from zero. d is a whole number from 0 to 9, or `#VALUE!`.	`=ROUND(E6, 1)` gives 123.8
`POW(x, n)`	The same as `x^n`.	`=POW(2, -3)` gives 0.125
`SQRT(x)`	The square root, correct to 4 decimal places.	`=SQRT(2)` gives 1.4142
`MOD(a, b)`	a − b × (a ÷ b rounded down), so the result has the sign of b.	`=MOD(-7, 3)` gives 2
`IF(test, a, b)`	a if test is not 0, otherwise b. All three are worked out first, so an error in any of them is the result.	`=IF(E2 > 80, "PASS", "FAIL")` gives FAIL
`LEN(x)`	How many characters x's shown text has.	`=LEN(A4)` gives 20
`UPPER(x)`	x's shown text with a to z made capitals.	`=UPPER(A2)` gives OMAR

Good luck, you'll need it!